GESTURE VOCALIZER GLOVE

MOHAMMED ABRAR
CHILVOJU MADHURI
RICHA MISHRA

Department of Electronics and Communication Engineering

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)
(Affiliated to Jawaharlal Nehru Technological University, Hyderabad, T.S.)

Chaitanya Bharathi P.O., Gandipet, Hyderabad – 500 075

2027

A WEARABLE SOLUTION FOR INCLUSIVE COMMUNICATION USING GESTURE TO VOICE TECHNOLOGY

REAL TIME PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT

OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

BY

MOHAMMED ABRAR (23261A0445)
CHILVOJU MADHURI (23261A0417)
RICHA MISHRA (23261A0452)

Department of Electronics and Communication Engineering

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Jawaharlal Nehru Technological University, Hyderabad, T.S.)

Chaitanya Bharathi P.O., Gandipet, Hyderabad – 500 075

2027

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Jawaharlal Nehru Technological University, Hyderabad, T.S.)

Chaitanya Bharathi P.O., Gandipet, Hyderabad-500 075

Department of Electronics and Communication Engineering

CERTIFICATE

Date: 01 June 2025

This is to certify that the Real time project work entitled "**Gesture Vocalizer Glove**" is a Bonafide work carried out by

Mohammed Abrar (23261A0445)

Chilvoju Madhuri (23261A0417)

Richa Mishra (23261A0452)

in partial fulfillment of the requirements for the degree of **BACHELOR OF TECHNOLOGY** in **ELECTRONICS & COMMUNICATION ENGINEERING** by the Jawaharlal Nehru Technological University, Hyderabad during the academic year 2024-25.

The results embodied in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

Professor S P Singh	**Dr. S P Singh**
Faculty Advisor	**Professor & Head**

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our Guide Dr. S.P. Singh, Head of Department of Electronics and Communication Engineering, M.G.I.T. (A), for his invaluable guidance, encouragement and necessary feedback in carrying out our Project.

We are highly indebted to our Faculty Advisor Ms. Archana Yadav, Assistant Professor, Electronics and Communication Engineering Department, who has given us all the necessary technical guidance in carrying out this Project.

We wish to express our sincere thanks to Dr. S.P. Singh, Head of the Department of Electronics and Communication Engineering, M.G.I.T. (A), for permitting us to pursue our Project, and encouraging us throughout the Project.

Finally, we thank all the people who have directly or indirectly helped us through the course of our Project.

Mohammed Abrar
Chilvoju Madhuri
Richa Mishra

Table of contents

CERTIFICATE FROM ECE DEPARTMENT (iii)

ACKNOWLEDGEMENTS (iv)

LIST OF FIGURES (vi)

LIST OF TABLES (vii)

ABSTRACT (viii)

CHAPTER 1. OVERVIEW

1.1 Aim of Project 1

1.2 Methodology 1

1.3 Significance and Applications 3

CHAPTER 2. MODULES AND WORKFLOW

2.1 Workflow Diagram 5

2.2 Functional Modules 6

2.2.1 List of Functional Modules 6

2.2.2 Working Code 10

CHAPTER 3. OUTPUT

3.1 Gesture Outputs 19

CHAPTER 4. PERFORMANCE EVALUATION AND ACCURACY ANALYSIS

4.1 Introduction 23

4.2 Gesture Accuracy 23

4.3 Result 24

4.7 Conclusions and Future Scope of Improvement 24

REFERENCES 26

LIST OF FIGURES

2.1 Gesture Vocalizer Glove 2

2.2 Circuit Diagram 3

2.3 Workflow Diagram 5

LIST OF TABLES

2.1 Modules 6

2.2 Gesture Accuracy 23

ABSTRACT

The Gesture Vocalizer Glove is an innovative wearable assistive device designed to translate hand gestures—particularly commonly used phrases—into audible speech. It aims to bridge the communication gap between individuals with speech and hearing impairments and the wider population who may not understand sign language. This barrier, often encountered in educational institutions, workplaces, healthcare settings, and public spaces, limits inclusivity and equal participation in daily interactions. At the core of the system is a sensor-integrated glove that combines five flex sensors, each attached to a finger, to detect bending and finger positions, with an MPU6050 inertial measurement unit (IMU) that captures motion data such as acceleration and orientation. These inputs provide a comprehensive understanding of the hand's posture and dynamics. The signals are processed by an ESP32 microcontroller, which interprets the gestures based on a predefined dataset of sign language representations.

One of the key features of the glove is its real-time gesture recognition capability. Once a gesture is identified, it is mapped to a corresponding word or phrase, which is then converted into speech using a web-based text-to-speech (TTS) API. The ESP32 hosts a web server interface that facilitates both gesture recognition feedback and voice output, allowing users or listeners to access the translated speech via any smartphone or browser-connected device on the same local network. The gesture dataset is customizable, allowing training on a wider range of gestures or regional sign languages. The design is low-cost and energy-efficient, utilizing readily available components such as the ESP32, flex sensors, and MPU6050. Importantly, the system operates without dependency on cloud processing, ensuring minimal latency and faster response time.

Developed using the Arduino IDE, this project recognizes ten distinct hand gestures to generate voice outputs for ten basic daily use phrases, demonstrating a practical blend of embedded systems, sensor fusion, and speech synthesis. It is scalable, adaptable, and can serve as a stepping stone for more sophisticated gesture-controlled communication aids in the future. By enabling instantaneous translation of gestures into speech, the Gesture Vocalizer Glove enhances the independence, confidence, and social participation of individuals with speech and hearing disabilities. It is a step toward building a more inclusive society where everyone can communicate and be heard.

INTRODUCTION

Communication forms the foundation of human interaction. Whether verbal or non-verbal, it enables individuals to express thoughts, emotions, needs, and ideas. However, for individuals with speech or hearing impairments, communication can often be a significant challenge. Traditional modes of interaction—such as spoken language or auditory cues—are not always accessible or effective, leading to social isolation, misunderstandings, and reduced opportunities in education, employment, and public life.

Sign language serves as a primary mode of communication for many people in the Deaf and hard-of-hearing communities. It is a rich, expressive language that uses hand gestures, facial expressions, and body movements to convey meaning. Despite its effectiveness, sign language is not universally understood by the general population. Most people, including those in public services, education, and healthcare sectors, lack training or familiarity with sign language. This language gap often leaves individuals with speech or hearing disabilities dependent on interpreters, written communication, or other external aids—none of which are always available or practical in real-time situations.

This disparity highlights a pressing need for assistive technologies that can bridge the communication divide and foster greater inclusivity. The **"Gesture Vocalizer Glove"** is one such innovation—a wearable device designed to recognize specific hand gestures based on sign language alphabets and phrases and convert them into audible speech. The glove is equipped with **flex sensors** attached to the fingers and an **MPU6050 motion sensor** on the wrist. These sensors collect data about finger bends and hand orientation, which are processed by an **ESP32 microcontroller**.

What sets this project apart is its simplicity and accessibility. The gesture recognition logic runs entirely on the ESP32, which also hosts a local **web server**. Through this server, a **user-friendly web interface** is presented that displays the detected gesture and uses the browser's built-in **text-to-speech (TTS)** capabilities to generate voice output—requiring no additional hardware for sound.

By enabling real-time gesture-to-speech translation in a compact, cost-effective, and portable format, the Gesture Vocalizer Glove empowers individuals with speech and hearing impairments to communicate more freely and confidently in diverse environments. It promotes **inclusivity** by reducing dependency on others for translation and breaking down communication barriers in everyday interactions. Ultimately, it fosters a more equitable society where everyone has a voice and the means to be heard.

OVERVIEW

Aim of Project

To design and build a hand-wearable device that can recognize hand gestures and convert them into speech output for real time communication

Methodology

Hardware Components used

1. ESP 32 Dev Board
2. Flex sensors – (5)
3. MPU6050 Sensor
4. Zero PCB
5. Jumper wires
6. Fabric Glove

Software Tools used

1. Arduino IDE (to write and upload code to microcontroller)
2. ESP32 Core for Arduino (to program ESP32 in Arduino environment)
3. WebServer Library (to set-up and handle HTTP routes for speech output)
4. Wi-Fi Library (to connect ESP32 to local network)
5. HTML, CSS, JavaScript (for building web-based user interface)
6. Web Speech API (API for text-to-speech) **Implementation**

1. **Sensor Integration:** The Gesture Vocalizer Glove utilizes five flex sensors and one MPU6050 inertial measurement unit (IMU). Each flex sensor is attached to a separate finger of the glove and connected to the analog input pins of the ESP32 microcontroller. These sensors change resistance when bent, which the ESP32 reads as analog voltage values. The MPU6050 sensor, which combines a 3-axis accelerometer and gyroscope, is connected to the ESP32 via the I2C communication protocol using the default SDA (GPIO21) and SCL (GPIO22) pins. This sensor setup provides both finger posture and hand orientation/movement data necessary for reliable gesture detection.
2. **Data Acquisition and Calibration:** Once the hardware was set up, data acquisition involved reading the analog values from the flex sensors and digital data from the MPU6050. Initial calibration was performed to determine the sensor value ranges

corresponding to common finger bending positions and hand orientations. For the flex sensors, the analog voltage was mapped to resistance levels to identify the degree of finger bending. For the MPU6050, raw accelerometer and gyroscope data were processed to calculate pitch, roll, and yaw angles, helping to detect dynamic hand movements and positions. These calibrated values form the baseline for recognizing specific gestures.

3. **Gesture Recognition:** A gesture recognition logic module was implemented in the ESP32 firmware. It continuously compares incoming sensor data against a predefined dataset of gestures. Each gesture is defined by a unique combination of flex sensor bend thresholds and MPU6050 orientation parameters. When the current readings closely match the defined parameters for a gesture, the corresponding label (e.g., “Hello”, “Thank you”) is identified. The recognition algorithm ensures noise filtering and applies a debounce technique to avoid false detections caused by transient hand movements.
4. **Web Server and UI:** The ESP32 was programmed to host a lightweight HTTP web server using the Arduino core for ESP32. The server serves a simple HTML-based web interface accessible from any device connected to the same Wi-Fi network. The UI displays the currently recognized gesture in text format. Additionally, the server exposes specific endpoints that allow clients (like a browser) to fetch updated gesture data dynamically using JavaScript. This ensures a seamless user experience without requiring page reloads.
5. **Speech Output:** For generating speech output, the system uses the web browser’s builtin **SpeechSynthesis API**, a part of the Web Speech API. When a gesture is detected and its text label is updated on the UI, JavaScript triggers this API to synthesize and play the corresponding speech directly through the browser on a phone or PC.

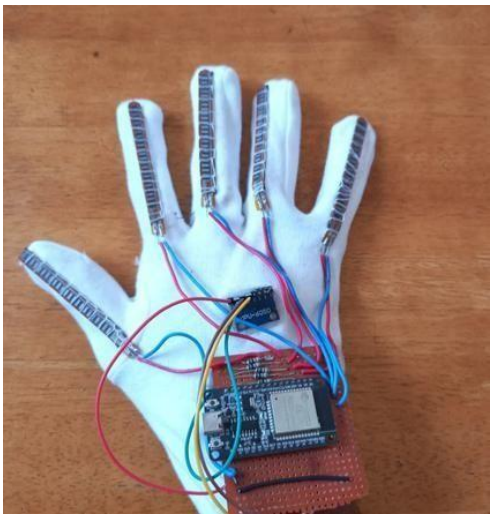


fig.1- Gesture Vocalizer Glove

Circuit Diagram

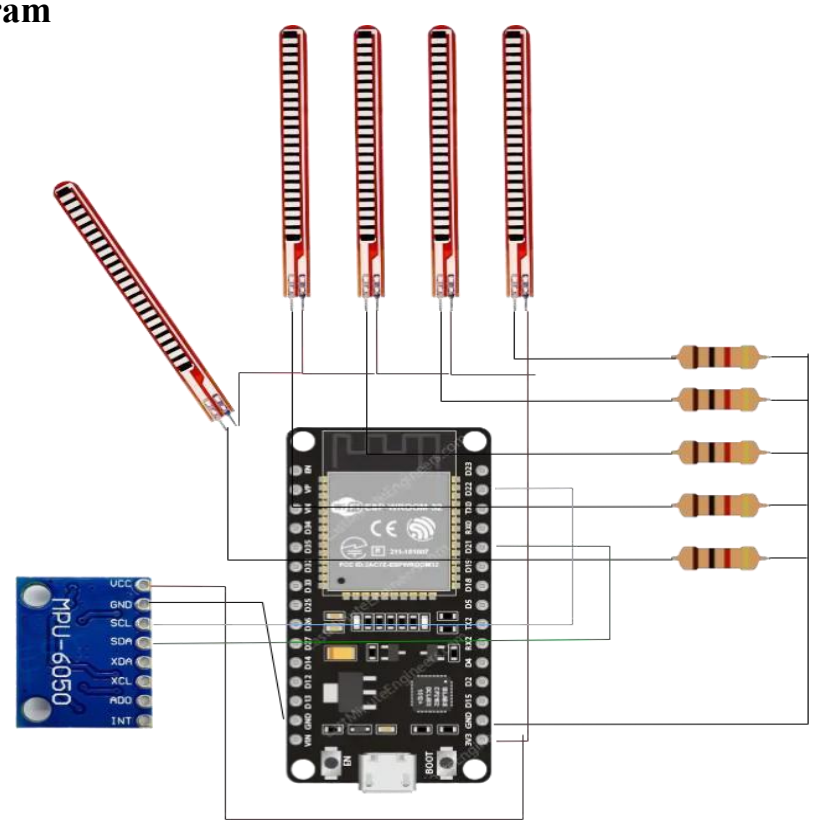


fig.2- Circuit Diagram

Significance and Applications

1. **For Individuals with hearing and speech impairments**
   - Barrier-free Communication: Translates sign language gestures into speech, enabling users to communicate with those who don't understand sign language.
   - Real-Time Interaction: Facilitates natural and immediate conversations in social, professional, and public settings.

2. **In Healthcare**
   - Assistive for Non-Verbal patients: Useful for patients with temporary speech loss due to surgery, trauma, or neurological conditions.
   - Improved Patient-Caregiver Interaction: Allows patients to express basic needs (e.g., pain, hunger, discomfort), improving quality of care.

3. **In Education**
    - Interactive Learning Tool: Helps in teaching sign language with visual and auditory feedback, making learning more engaging.
    - Inclusive Classrooms: Supports inclusive education by enabling better interaction between hearing-impaired students and peers/teachers.
    - Skill Development: Encourages the use of assistive tech among students and introduces embedded systems and gesture recognition concepts.

4. **In Human-Computer Interaction**
    - In Gesture-Controlled Interfaces: Can be adapted for controlling electronic devices, appliances, or user interfaces through hand movements.
    - Wearable Computing: Represents a step forward in creating intuitive, sensorbased wearable technology.
    - Accessibility Innovations: Lays a foundation for future interfaces that don't rely on voice or touch, enhancing accessibility.

5. **Public and Social Services**
    - Assistive Technology in Public Spaces: Could be installed in service centers, railway stations, or banks to assist hearing-impaired users.
    - Customer Service Applications: Allows easier interaction with support personnel who may not know sign language.

MODULES AND WORKFLOW

Introduction

This section outlines the overall workflow of the Gesture Vocalizer Glove and provides a detailed explanation of each functional module within the code, from reading sensor data and recognizing gestures to hosting a web interface and generating voice outputs. Together, these modules form an integrated system that captures hand gestures in real time, processes them using sensor data, and translates them into audible speech through a web-based interface. Workflow Diagram

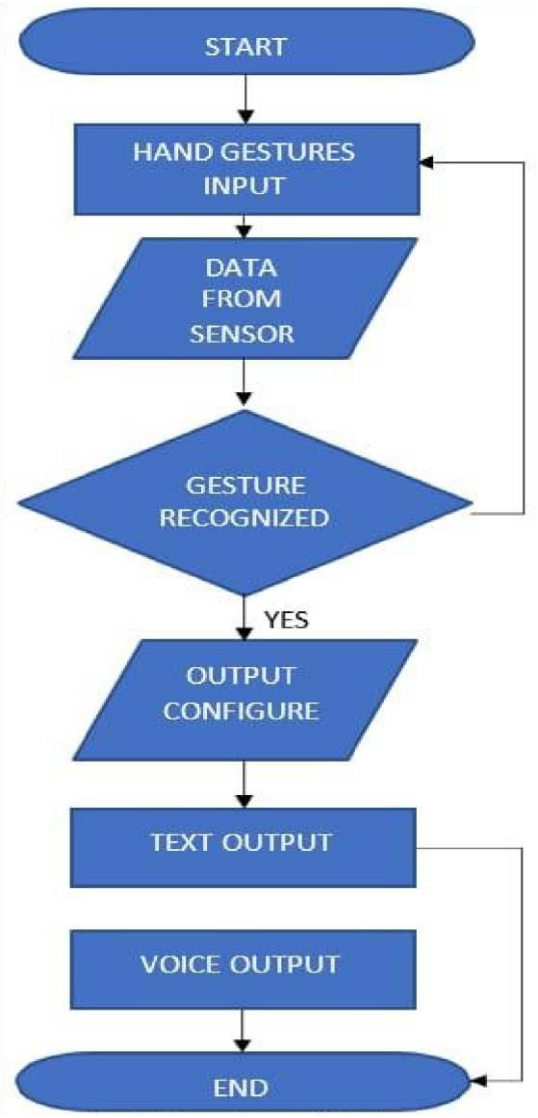


fig.3 – Workflow Chart

Functional Modules

List of Functional Modules

Module Name	Functionality
Wi-Fi & Web Server	Connectivity and hosting UI
Sensor Integration	Reads flex sensors and MPU6050
Sensor Calibration	Establishes baseline resistance
Gesture Recognition Logic	Maps sensor data to gestures
Sensor Reading	Continuously reads and processes sensor data
Web UI	HTML + JavaScript for user interaction
Gesture Display & TTS Output	Updates browser UI and triggers voice output
Main Loop	Coordinates all above tasks in the runtime loop

table 1- Modules

1. **Wi-Fi and Web server:**

Purpose: Enables connectivity and serves the UI via HTTP.

- The ESP32 connects to a Wi-Fi network using the provided SSID and password.
- A local **Webserver** is initialized on port 80.
- The **mDNS responder** allows accessing the glove via a friendly URL like

  `http://gestureglove.local`.
- The server handles routes such as:

- `/` – Serves the HTML UI. o `/gesture` – Returns the current gesture text. o `/speak` – Triggers a voice output request (handled on the client side).
- `/tts` – Placeholder for future text-to-speech expansion

2. Sensor Integration Module:

Purpose: Reads data from hardware sensors.

- **Flex Sensors** are connected to the analog pins of the ESP32 to measure finger bending by monitoring resistance.
- The **MPU6050** sensor is connected via I2C (SDA/SCL) to measure the orientation (tilt) of the hand using accelerometer data.
- These hardware components provide the raw input necessary for gesture recognition.

3. Sensor Calibration Module:

Purpose: Establishes baseline values for accurate gesture detection.

- On startup, the glove performs calibration to measure **baseline resistance** of each flex sensor when the hand is relaxed.
- *calibrateSensors()* takes 10 readings for each flex sensor and averages them.
- These baselines are used later to calculate **resistance ratios**, which determine how much each finger is bent.

4. Gesture Recognition Logic:

Purpose: Matches sensor values to specific gestures.

- Based on the real-time sensor ratios and hand angle, the function *checkGestures()* evaluates if a combination matches any predefined gesture pattern.
- Each gesture has its own threshold conditions (e.g., which fingers are bent/straight, and tilt angle).

- If a match is found, a **corresponding phrase** (e.g., “Hello, how are you?”) is assigned to *lastDetectedGesture.*

5. **Sensor Reading:**

Purpose: Continuously reads and processes sensor data.

- The *readFlexSensors()* function:
  - Reads analog values from the pins. ○ Converts them to resistance using a voltage divider formula. ○ Computes **resistance ratio** with respect to the baseline.
- The *readAngleSensor()* function retrieves accelerometer data from MPU6050 by calculating **tilt angle** using trigonometric operations.

6. **Web UI Module:**

Purpose: Allows users to interact with the glove.

- Served via the root (`/`) endpoint.
- HTML, CSS, and JavaScript are embedded in the code.
- UI Components: ○ Displays the detected gesture in real-time. ○ A **Refresh** button to manually get the gesture. ○ A **Speak** button to trigger the browser’s speech synthesis.
- Simple and responsive design for easy access on desktop and mobile.

7. **Gesture Display and TTS Output Module:**

Purpose: Provides real-time visual and auditory feedback.

- The JavaScript on the web page:
  - Uses `fetch()` every second to get the current gesture from `/gesture`. ○ Displays the gesture text on the page.

- When the Speak button is clicked, it triggers the **SpeechSynthesis API** to **speak the gesture aloud**.

- This ensures real-time, **audible feedback**, even though no physical speaker is attached to the glove.

8. Main Loop:

Purpose: Coordinates all modules in runtime.

- The `loop()` function in Arduino continuously:
  - Handles HTTP client requests via `server.handleClient()`.
  - Reads sensor values (`readFlexSensors`, `readAngleSensor`). o Displays the current readings to the Serial Monitor for debugging.

Working Code

```
#include <Wire.h>
#include <MPU6050.h>
#include <WiFi.h>
#include <WebServer.h>
#include <ESPmDNS.h>
#include <HTTPClient.h>

// WiFi Credentials - replace with your
own const char* ssid = "F22";
const char* password = "mdab23092004";

// Web Server setup
WebServer server(80);

// Define pins for flex sensors  const int flexPins[] =
{32,35,34,39,36};  // ADC pins on
ESP32 const int numSensors = 5;

// Variables for sensor
readings float
flexValues[numSensors]; float
baselineValues[numSensor
s];
const float BRIDGE_RESISTANCE = 10000.0;  // 10k bridge resistance

// MPU6050 object  MPU6050
mpu;

// Struct to name parameters more
explicitly struct GestureParameters {
  float index;
float
middle
;   float
ring;   float
pinky;
float
thumb;
```

```
float
angle;
};

GestureParameters currentGesture;
String lastDetectedGesture = "None";

// Function prototypes to resolve
compilation issues void
calibrateSensors(); void handleGesture(); void
handleSpeak(); void handleTTS();
void handleRoot();

void setup() {
Serial.begin(115200);

  // Initialize WiFi
  WiFi.begin(ssid, password);   while
(WiFi.status() != WL_CONNECTED)
{     delay(500);     Serial.print(".");
  }
  Serial.println("");
  Serial.print("Connected to WiFi. IP address: ");
  Serial.println(WiFi.localIP());

  // Setup mDNS responder   if
(MDNS.begin("gestureglove")) {     Serial.println("MDNS
responder
started");
  }

  // Initialize I2C for MPU6050
  Wire.begin();

  // Initialize
MPU6050
mpu.initialize();   if
(!mpu.testConnectio
n()) {
    Serial.println("MPU6050 connection failed");
while(1);
```

```
  }

  // Initialize ADC
  analogReadResolution(12);  // ESP32 has 12-bit ADC

  // Calibration phase
  Serial.println("Calibrating sensors... Keep fingers straight and hand steady");
calibrateSensors();
  Serial.println("Calibration complete!");

  // Setup Web Server Routes
server.on("/", handleRoot);
server.on("/gesture", handleGesture);
server.on("/speak", handleSpeak);
  server.on("/tts", handleTTS);

  // Start server
server.begin();
Serial.println("HTTP server
started");
}

void handleRoot() {
  String html = String("<!DOCTYPE html>") +
    "<html lang=\"en\">" +
    "<head>" +
      "<meta charset=\"UTF-8\">" +
      "<meta name=\"viewport\" content=\"width=device-width,
initialscale=1.0\">" +
      "<title>Gesture Glove Interface</title>" +
      "<style>" +
        "body { font-family: Arial, sans-serif; background-color: #f0f0f0; display:
flex; justify-content: center; align-items: center; height: 100vh; margin: 0; }" +
        ".container { background-color: white; border-radius: 10px; box-shadow: 0
4px 6px rgba(0,0,0,0.1); padding: 20px; text-align: center; }" +
        ".gesture-display { font-size: 2em; margin: 20px; color: #333; }" +
        "button { background-color: #4CAF50; color: white; border: none; padding:
10px
20px; margin: 10px; border-radius: 5px; cursor: pointer; }" +
      "</style>" +
```

```
    "</head>" +
    "<body>" +
      "<div class=\"container\">" +
        "<h1>Gesture Glove</h1>" +
        "<div id=\"gestureDisplay\" class=\"gesture-display\">-</div>" +
        "<button onclick=\"fetchGesture()\">Refresh Gesture</button>" +
        "<button onclick=\"speakGesture()\">Speak Gesture</button>" +
      "</div>" +
      "<script>" +
        "function fetchGesture() {" +
          "fetch('/gesture')" +
            ".then(response => response.text())" +
            ".then(data => {" +
"document.getElementById('gestureDisplay').textContent = data;" +
            "});" +
        "}" +
        "function speakGesture() {" +
          "fetch('/gesture')" +
            ".then(response => response.text())" +
            ".then(data => {" +
              "if ('speechSynthesis' in window) {" +
                "let utterance = new SpeechSynthesisUtterance(data);" +
                "window.speechSynthesis.speak(utterance);" +
              "}" +
            "});" +
        "}" +
        "setInterval(fetchGesture, 1000);" +
      "</script>" +
    "</body>" +
    "</html>";

  server.send(200, "text/html", html);
}

void handleGesture() {    server.send(200,
"text/plain", lastDetectedGesture);  }

void handleSpeak() {
  server.send(200, "text/plain", "Speak request received for: " +
```

```
lastDetectedGesture); }

void handleTTS() {    server.send(200, "text/plain",
"TTS placeholder");  }

void loop() {
  // Handle web server clients
  server.handleClient();

  // Read and process all sensors
readFlexSensors();
  readAngleSensor();

  // Continuously display current ratios and angle
displayCurrentReadings();

  // Check for specific gestures
checkGestures();

  delay(700);  // Slight delay to prevent overwhelming  }

void calibrateSensors() {
  // Take multiple readings for more accurate
calibration   const int numReadings = 10;

  // Calibrate flex sensors
for(int i = 0; i < numSensors;
i++) {     float sum = 0;
    for(int j = 0; j < numReadings; j++) {
      sum += calculateResistance(analogRead(flexPins[i]));
delay(100);
    }
    baselineValues[i] = sum / numReadings;
  }
}

void readFlexSensors() {
for(int i = 0; i < numSensors;
i++) {
```

```
   // Calculate ratio of current resistance to baseline
float currentResistance =
calculateResistance(analogRead(flexPins[i]));     float ratio = currentResistance
/ baselineValues[i];

sensor index     switch(i) {       case 0:
currentGesture.thumb = ratio; break;       case
1: currentGesture.index = ratio; break;
case 2: currentGesture.middle = ratio; break;
case 3: currentGesture.ring = ratio; break;
     case 4: currentGesture.pinky = ratio; break;
   }
  }
}
void
readAngleSensor() {
// Read raw
accelerometer data
int16_t ax, ay, az;
  mpu.getAcceleration(&ax, &ay, &az);

  // Calculate angle in degrees using
arctangent   // This calculates the angle from
the horizontal plane
  float angle = atan2(sqrt(ax*ax + ay*ay), az) * 180.0 / PI;

  // Store the absolute angle
currentGesture.angle = abs(angle);
}

void displayCurrentReadings() {
Serial.println("Current Readings:");
  Serial.print("Thumb: "); Serial.print(currentGesture.thumb);
  Serial.print(" | Index: "); Serial.print(currentGesture.index);
  Serial.print(" | Middle: "); Serial.print(currentGesture.middle);
  Serial.print(" | Ring: "); Serial.print(currentGesture.ring);
  Serial.print(" | Pinky: "); Serial.print(currentGesture.pinky);
  Serial.print(" | Angle: "); Serial.println(currentGesture.angle);  }

float calculateResistance(int adcValue) {
  float voltage = (adcValue * 3.3) / 4096.0;  // Convert ADC to
```

```
voltage   float flexResistance = BRIDGE_RESISTANCE * (3.3 / voltage
- 1.0);   return flexResistance;
}

void checkGestures() {
  // All fingers bent    bool gesture1 =
(currentGesture.index > 1.05 &&
currentGesture.middle > 1.05 &&
currentGesture.ring > 1.05 &&
currentGesture.pinky > 1.05 &&
currentGesture.thumb > 1.05 &&
currentGesture.angle >= 75 && currentGesture.angle <= 100);

  // Gesture A: 4 fingers (except thumb) bent, thumb
extended   bool gesture2 = (
currentGesture.index > 1.05 &&
currentGesture.middle > 1.05 &&
currentGesture.ring > 1.05 &&
currentGesture.pinky > 1.05 &&
currentGesture.thumb < 1.05 &&
currentGesture.angle >= 75 && currentGesture.angle <= 100);

  // Gesture B: Index and thumb straight, others
bent   bool gesture3 = (
currentGesture.index < 1.05 &&
currentGesture.middle >
1.05 && currentGesture.ring > 1.05
&& currentGesture.pinky > 1.05
&& currentGesture.thumb < 1.05
&&  currentGesture.angle >= 75 && currentGesture.angle <= 100);

  // Only index and thumb finger bent
bool gesture4 = (
currentGesture.index > 1.05 &&
currentGesture.middle < 1.05 &&
currentGesture.ring < 1.05 &&
currentGesture.pinky < 1.05 &&
currentGesture.thumb > 1.05 &&
currentGesture.angle >= 75 && currentGesture.angle <= 100);
```

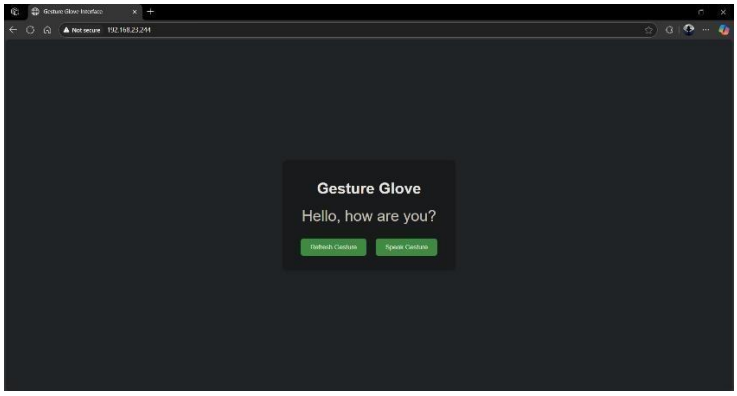
```
  // Gesture D: Index and middle straight others
bent   bool gesture5 = (currentGesture.index < 1.05
&& currentGesture.middle < 1.05
&& currentGesture.ring > 1.06
&& currentGesture.pinky > 1.05
&& currentGesture.thumb < 1.05
&&  currentGesture.angle >= 75 && currentGesture.angle <= 100);

  // middle and ring bent others straight
bool gesture6 = (
currentGesture.index < 1.05 &&
currentGesture.middle > 1.05 &&
currentGesture.ring > 1.05 &&
currentGesture.pinky < 1.05 &&
currentGesture.thumb > 1.05 &&
currentGesture.angle >= 75 && currentGesture.angle <= 100);

  // all fingers bent except pinky   bool
gesture7 = (currentGesture.index > 1.05 &&
currentGesture.middle > 1.05 &&
currentGesture.ring > 1.05 &&
currentGesture.pinky < 1.03 &&
currentGesture.thumb > 1.05 &&
currentGesture.angle >= 75 && currentGesture.angle <= 100);

  // all fingers bent except index   bool
gesture8 = (
currentGesture.index < 1.05 &&
currentGesture.middle > 1.05 &&
currentGesture.ring > 1.05 &&
currentGesture.pinky > 1.05 &&
currentGesture.thumb > 1.05 &&
currentGesture.angle >= 75 && currentGesture.angle <= 100);

  // all fingers open    bool gesture9 =
(currentGesture.index < 1.05 &&
currentGesture.middle < 1.05 &&
currentGesture.ring < 1.05 &&
currentGesture.pinky < 1.05 &&
currentGesture.thumb < 1.05 &&
currentGesture.angle >= 75 && currentGesture.angle <= 100);
```

```
  bool gesture10 = (
currentGesture.index > 1.05 &&
currentGesture.middle > 1.05 &&
currentGesture.ring > 1.05 &&
currentGesture.pinky > 1.05 &&
currentGesture.thumb < 1.05 &&
currentGesture.angle >= 130 && currentGesture.angle <= 170);   //
Update last detected gesture (priority order if multiple match)    if
(gesture1) {
    lastDetectedGesture = "Hello, how are you?";
Serial.println("Detected Gesture: Hello, how are you?");
} else if (gesture2) {
    lastDetectedGesture = "I need help";
Serial.println("Detected Gesture: I need help");
  } else if (gesture3) {
    lastDetectedGesture = "Thank you";
Serial.println("Detected Gesture: Thank you");
  } else if (gesture4) {
    lastDetectedGesture = "Yes, please";
Serial.println("Detected Gesture: Yes, please");
  } else if (gesture5) {
lastDetectedGesture = "No";
Serial.println("Detected Gesture: No");
  }  else if (gesture6) {
lastDetectedGesture = "Bye";
Serial.println("Detected Gesture: Bye");
  } else if (gesture7) {
    lastDetectedGesture = "Excuse me";
Serial.println("Detected Gesture: Excuse me");
  } else if (gesture8) {
    lastDetectedGesture = "Wait a moment";
Serial.println("Detected Gesture: Wait a moment");
  } else if (gesture9) {
    lastDetectedGesture = "Bless you";
Serial.println("Detected Gesture: Bless you");
  } else if (gesture10) {
lastDetectedGesture = "Sure";
Serial.println("Detected Gesture:
Sure");    } else {
lastDetectedGesture = "None";
  }
 }
```

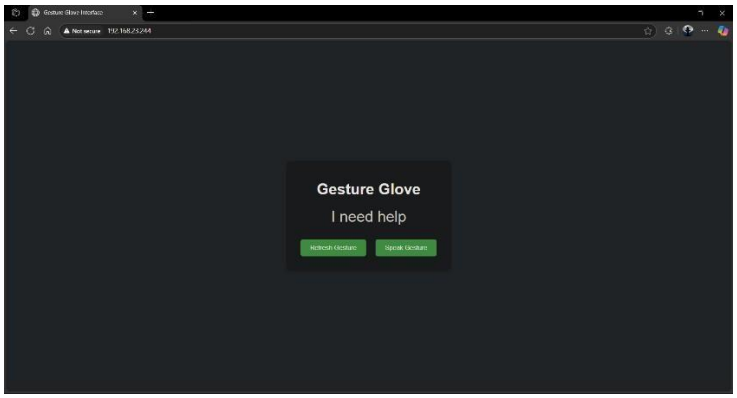
OUTPUT

Gesture Outputs

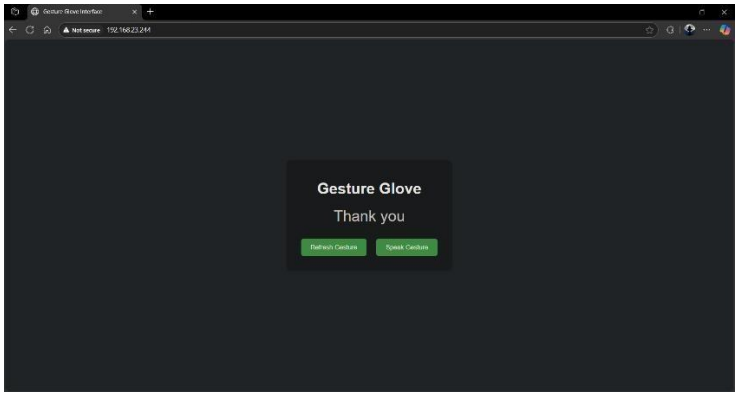
1. Gesture A



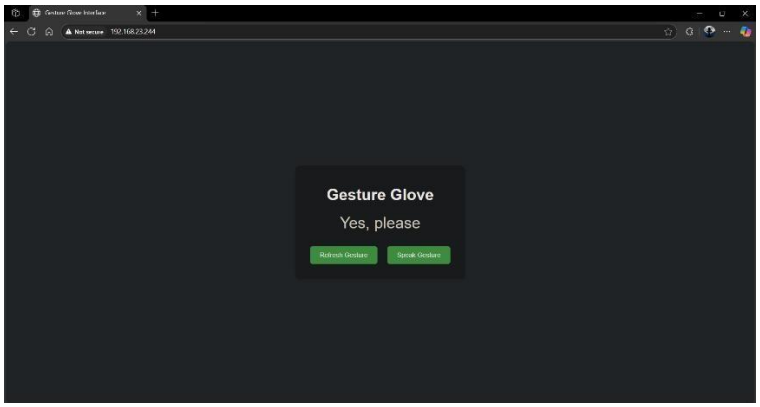
2. Gesture B



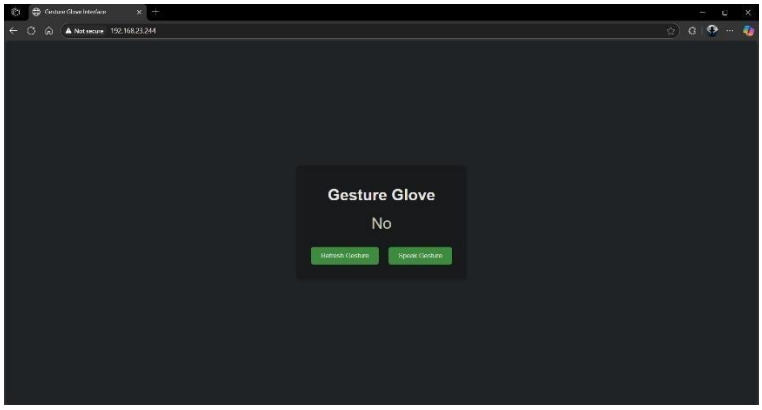
3. Gesture C



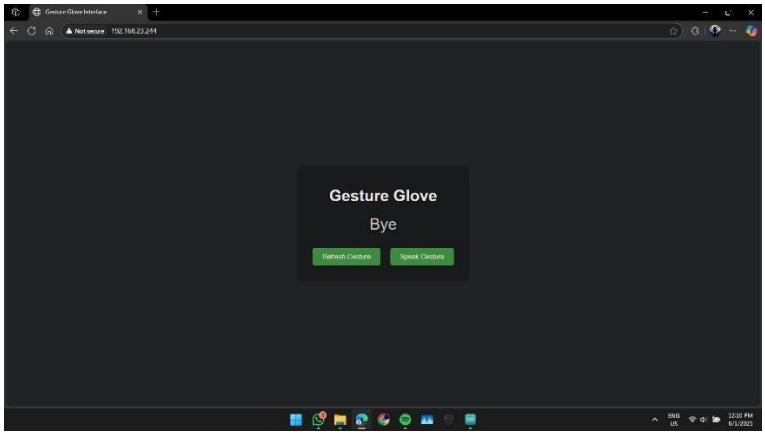
4. Gesture D



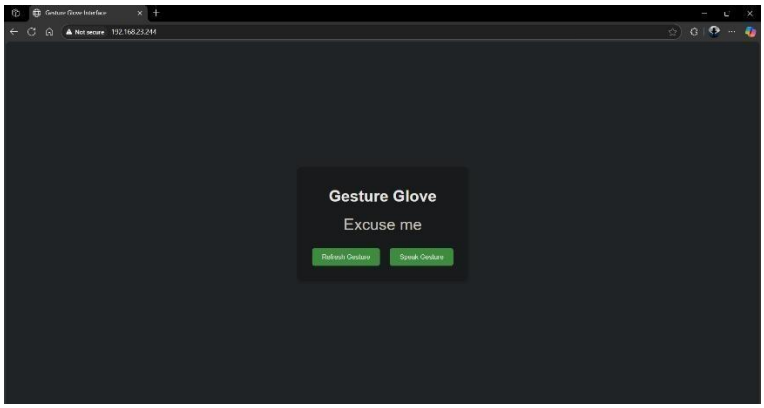
5. Gesture E



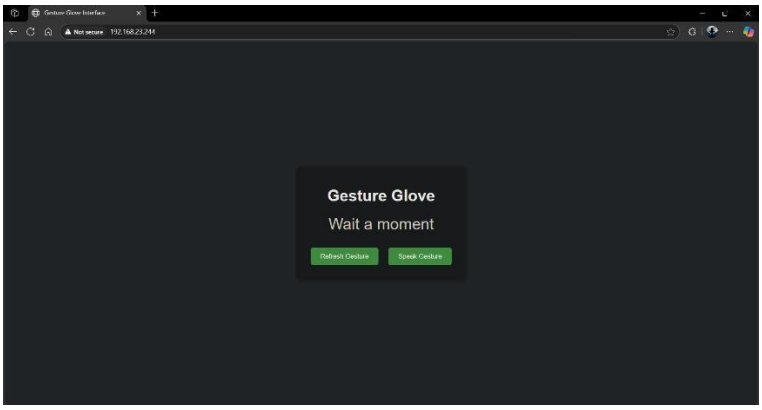
6. Gesture F



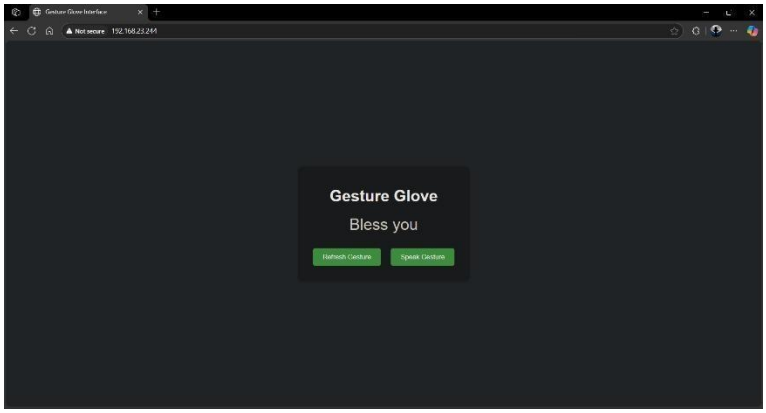
7. Gesture G



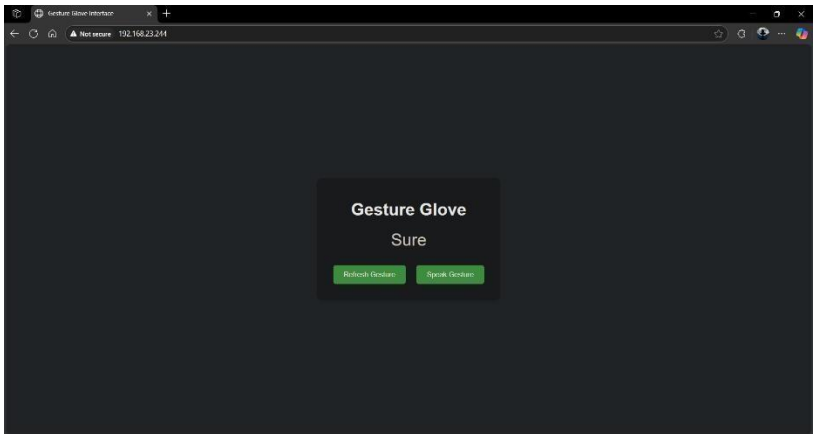
8. Gesture H



9. Gesture I



10. Gesture J


Gesture Glove Interface
Not secure
192.168.23.244
Gesture Glove
Sure
Refresh Gesture
Speak Gesture

PERFORMANCE EVALUATION

Introduction

The primary aim of this section is to assess how accurately the system recognizes and responds to predefined hand gestures, ensuring it functions reliably as an assistive communication tool. The evaluation focuses on measuring the system's gesture detection accuracy, identifying any misclassifications, and analyzing sources of error. Each gesture was tested repeatedly in varied conditions, and the number of correct recognitions was recorded to calculate the detection accuracy. A performance table summarizing these results has been included to provide a clear view of the system's effectiveness and reliability. This analysis not only validates the functionality of the glove but also highlights areas for further improvement and optimization in future.

Gesture Accuracy

S.No.	Gesture Description	Voice Output	Trials Performed	Correctly Detected	Detection Accuracy (%)
1	All fingers bent	Hello, how are you?	20	18	90%
2	Thumb extended, all other fingers bent	I need help	20	18	90%
3	Thumb and index straight, others bent	Thank you	20	16	80%
4	Only index and thumb bent	Yes, please	20	15	75%
5	Index and middle straight, others bent	No	20	18	90%
6	Middle and ring bent, others straight	Bye	20	14	70%
7	All fingers bent except last finger	Excuse me	20	17	85%
8	All fingers bent except index	Wait a moment	20	17	85%
9	All fingers straight	Bless you	20	15	75%
10	All fingers straight (tilted horizontally)	Sure	20	13	65%

- Hand used – **Right Hand**

- Accuracy = $\frac{Correct\ detected}{No\ of\ trials\ \ perfomed} \times 100\%$

Results

- The Gesture Vocalizer Glove was evaluated using **10 distinct hand gestures**, each representing a basic daily-use phrase.
- **20 trials were conducted** for each gesture using the **right hand**.
- **High accuracy gestures**:
    - "Hello, how are you?", "I need help", and "No" each had a **90% detection accuracy**.
    - "Excuse me" and "Wait a moment" followed with **85% accuracy**.
- **Lower accuracy gestures**: ○ "Bye" – **70% accuracy** ○ "Sure" – **65% accuracy**
- **Reasons for lower accuracy**:
    - Gestures with **subtle finger positions or wrist orientations** were harder to differentiate.
    - **Sensor sensitivity and minor variations** in hand posture contributed to misclassifications.
- **Average detection accuracy** across all gestures: **81.5%**

Conclusion and Future Scope for Improvement

Conclusion

The Gesture Vocalizer Glove successfully demonstrates a low-cost, real-time assistive solution for translating hand gestures into voice outputs, aimed primarily at supporting individuals with speech and hearing impairments. The system achieves a commendable average gesture recognition accuracy of 81.5%, validating its reliability in interpreting predefined sign gestures. Its integration of flex sensors, an MPU6050 sensor, and a web-based TTS interface showcases the feasibility of wearable, IoT-based communication tools in assistive technology. While certain complex gestures showed reduced accuracy, the overall system performed as expected and offers a strong foundation for future enhancements.

Future Scope for improvement

- **Increased Gesture Dataset**: Expand the number of recognizable gestures to cover a wider vocabulary, including full alphabets and common phrases.

- **Machine Learning Integration**: Use machine learning models to dynamically classify gestures, reducing dependency on manually defined thresholds.

- **Dynamic Calibration**: Implement real-time adaptive calibration to accommodate different hand sizes and sensor drift over time.

- **Multi-Language Support**: Add multi-language voice outputs for diverse user demographics.

- **Mobile App Integration**: Develop a companion mobile application for offline access, voice playback, and custom gesture configuration.

- **Two-Handed Gesture Support**: Extend the system to recognize two-handed gestures used in full sign languages.

- **Haptic Feedback**: Include vibration motors to give tactile feedback when gestures are successfully detected or misunderstood.

- **Enhanced Wearability**: Improve the comfort, flexibility, and form factor of the glove for long-term daily use.

- **Noise Filtering & Error Correction**: Implement algorithms for sensor noise reduction and automatic correction of near-match gestures.

- **Battery Optimization**: Optimize power usage and consider wireless charging for extended usage and portability

References:

Datasheets and Components:

1. Espressif Systems, "ESP32-WROOM-32 Datasheet," 2022. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

2. SparkFun Electronics, "Flex Sensor Datasheet," [Online]. Available: https://www.sparkfun.com/datasheets/Sensors/Flex/flex22.pdf

3. InvenSense, "MPU-6050 Product Specification," 2013. [Online]. Available: https://invensense.tdk.com/wpcontent/uploads/2015/02/MPU-6000-Datasheet1.pdf

Libraries and Software:

1. Arduino, “MPU6050 Library by Electronic Cats,” [Online]. Available: https://github.com/ElectronicCats/mpu6050
2. Arduino, “ESPAsyncWebServer Library,” [Online]. Available: https://github.com/me-nodev/ESPAsyncWebServer
3. Arduino, "ESP32 Core for Arduino," [Online]. Available: https://github.com/espressif/arduino-esp32